Lab 3 — Component Modelling & Architectural Pattern Selection

Project: Customizable Subscription Box Scheduler

---

1. Architecture Selection

Chosen architecture: Layered Architecture, with a client-server deployment topology sitting underneath it (not competing with it). Microservices and pure Client-Server are not the right modelling answer for this lab — see Section 3 for why.

2. Why This Architecture Fits Best

- The functionality is workflow-driven, not independently scalable. Every epic (Subscription Management, Box Customization, Fulfillment Processing) is a set of CRUD-plus-validation operations against shared subscriber and box data. No requirement calls for independent scaling or release cycles per epic.
- There is a clean, reusable business rule. The 48-hour swap cutoff and the pre-renewal pause/skip cutoff are the same kind of check applied in two different epics. A layered design isolates that rule in the business logic layer and lets both services call into it — directly reflecting the use case's alternate flow (changes rejected after cutoff).
- Two distinct actors, one coherent backend. Subscriber and Fulfillment Lead consume the same underlying business rules and data (a subscriber's saved selections become the Fulfillment Lead's "finalized selections"). Layering keeps that data and logic in one consistent place instead of forcing it through inter-service calls.
- Security and performance requirements are stated but unspecified. A layered structure gives natural, single points to enforce cross-cutting concerns without inventing infrastructure details that were not provided.

3. Why the Other Two Are Weaker Here

Microservices Architecture

Overkill, not wrong in principle, wrong for this scope. Nothing in the requirements calls for independently deployable services: no separate teams, no separate scaling needs, no polyglot persistence, no independent release cycles. Splitting Subscription Management, Box Customization, and Fulfillment Processing into separate services would force solving problems that are not present (distributed transactions across pause/skip/swap-then-fulfill, network reliability, service discovery) for no requirement-driven benefit.

Client-Server Architecture

True, but not sufficient as the architecture for this lab. Client-Server correctly describes the physical topology (Subscriber and Fulfillment Lead clients talking to a server), but says nothing about how the server is internally organized, which is the point of a component diagram. Using Client-Server alone would yield two boxes and a line, not the 5+ components and 4+ interfaces the lab requires. It is the deployment view; Layered is the internal structural view — they are not mutually exclusive, but only one answers "how is the system organized."

4. Component Breakdown

Component	Responsibility
Subscriber Portal	Subscriber-facing UI. Initiates create/pause/skip subscription and view/swap/save box actions.
Fulfillment Portal	Fulfillment Lead-facing UI. Initiates viewing finalized selections, generating the manifest, exporting labels.
Subscription Management Service	Business logic for Create Subscription, Pause Subscription, Skip Billing Cycle. Delegates timing checks to the Cutoff Policy Validator.
Box Customization Service	Business logic for View/Swap/Save monthly box items using preference tags. Delegates the 48-hour swap-window check to the Cutoff Policy Validator.
Cutoff Policy Validator	Shared rule component encapsulating the 48-hour swap cutoff and pre-renewal pause/skip cutoff logic; produces the accept/reject decision used in the alternate flow.
Fulfillment Processing Service	Business logic for View Finalized Selections, Generate Fulfillment Manifest, Export Shipping Labels. Reads locked, post-cutoff selections from the data store.
Data Store	Persistence layer for subscriptions, box selections, and fulfillment records.

Seven components in total — above the 5-component minimum. No separate Authentication/Authorization component was added: the brief mentions security requirements without specifics, and inventing an auth flow would add a feature not supported by the project context. It plugs in cleanly as an 8th component in front of both portals if your rubric requires it shown explicitly.

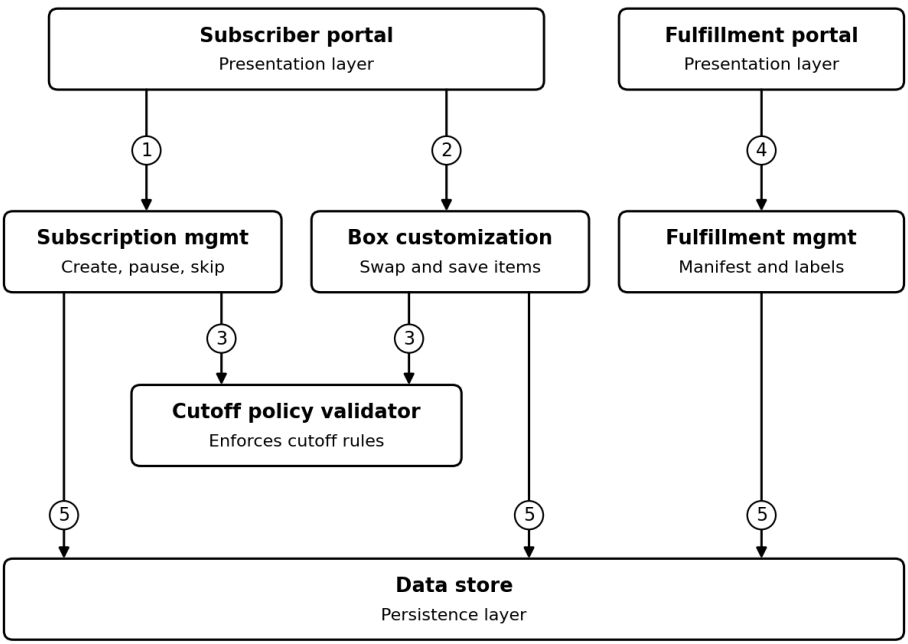
5. Interface Breakdown

#	Interface	Provided by	Required by	Covers
1	**ISubscriptionManagement**	Subscription Mgmt Service	Subscriber Portal	Create/pause/skip operations
2	**IBoxCustomization**	Box Customization Service	Subscriber Portal	View/swap/save operations
3	**ICutoffPolicy**	Cutoff Policy Validator	Subscription Mgmt Service, Box Customization Service	Accept/reject decision vs. 48h / renewal cutoff
4	**IFulfillmentProcessing**	Fulfillment Processing Service	Fulfillment Portal	View selections, generate manifest, export labels
5	**IPersistence**	Data Store	All three business services	Read/write subscription, box, fulfillment records

Five distinct interfaces across eight dependency links — above the 4-interface minimum.

6. Diagram Blueprint

Reproduce the diagram below as four horizontal tiers: (1) Subscriber Portal and Fulfillment Portal; (2) Subscription Management Service, Box Customization Service, Fulfillment Processing Service; (3) Cutoff Policy Validator, centered under Subscription Management and Box Customization only; (4) Data Store, spanning the full width. Draw each connection as a UML dependency arrow (open arrowhead pointing from consumer to provider). In draw.io, use the UML shape library's Component boxes, and represent each interface as a lollipop on the provider side and a socket on the consumer side if your rubric expects strict UML ball-and-socket notation.



Interface legend

1 ISubscriptionManagement — portal requires, subscription mgmt provides

2 IBoxCustomization — portal requires, box customization provides

3 ICutoffPolicy — mgmt and customization require, validator provides

4 IFulfillmentProcessing — fulfillment portal requires, fulfillment mgmt provides

5 IPersistence — all three services require, data store provides

Figure 1. UML component diagram — Customizable Subscription Box Scheduler (numbers reference the interface table in Section 5).